Chomsky: A Formal-Language-Theory Approach to Security Analysis in Code Repositories

Code Hazard Observation via Modeling of Syntax and KeY-patterns

Juan David Calderón · Andrés López · Jaime Muñoz

Universidad ICESI — Facultad de Ingeniería
Computación y Estructuras Discretas III

Abstract

Modern software repositories routinely contain security weaknesses introduced directly in source code and configuration files: hardcoded credentials, leaked secrets, exposed internal IP addresses, and insecure configuration values.

Chomsky is a code repository security analyzer that leverages four levels of the Chomsky hierarchy — regular expressions, deterministic finite automata, finite state transducers, and context-free grammars — to systematically **detect, classify, transform, and validate** security vulnerabilities before they reach production.

Problem

- Public repositories expose **thousands of API keys per week** on GitHub.
- Regex-only scanners produce **high false positives** and cannot track temporal relationships between events (credential declared on line 10, leaked on line 50).
- Nested configuration files cannot be validated with regular expressions: matched braces $\{^n\ w\ \}^n$ require unbounded memory, ruling out any finite automaton.

Objectives

1. Extract security-relevant tokens from source code using **regular expressions**.
2. Classify each file as `SAFE`, `NEEDS_REVIEW`, or `SECURITY_VIOLATION` using a **deterministic finite automaton**.
3. Automatically refactor insecure constructs using a **finite state transducer**.
4. Validate structured configurations against a **context-free grammar** that enforces security policies.

Module 1 — Detection

Regular Expressions (Chomsky Type-3)

Source files are treated as strings over Σ = `ASCII`. Seven regular languages detect security-relevant patterns; their union is itself regular by **Kleene's theorem**:

```
L(detector) = L(AWS_KEY) ∪ L(HARDCODED_CRED)
            ∪ L(PRINT_LEAK) ∪ L(CONSOLE_LEAK)
            ∪ L(IPv4) ∪ L(ENV_REF) ∪ L(TODO)
```

Pattern	Regex
AWS_KEY	`AKIA[0-9A-Z]{16}`
HARDCODED_CRED	`(password|secret)\s*=\s*".*?"`
IPv4	`\b(\d{1,3}\.){3}\d{1,3}\b`

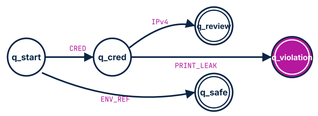
The detector emits a sequence of abstract tokens consumed by Module 2.

Module 2 — Classification

Deterministic Finite Automaton (DFA)

5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

- **Q** = { q_start, q_cred, q_review, q_violation, q_safe, q_sink }
- **Σ** = abstract tokens from Module 1
- $\mathbf{q_0}$ = q_start · **F** = { q_safe, q_review, q_violation }



Key insight: the DFA enforces temporal order — a credential must precede a leak for a file to be flagged as a violation. Built with `pyformlang`.

Module 3 — Transformation

Finite State Transducer (FST)

7-tuple $T = (Q, \Sigma, \Delta, \delta, q_0, F, \lambda)$ where:

- **Σ** (input) = abstract security tokens
- **Δ** (output) = { REWRITE_CRED, REMOVE_LEAK, FLAG_IP, PASSTHROUGH }
- **Non-determinism:** IPv4 → { FLAG_IP, PASSTHROUGH }, resolved by priority.

```
# Before
password = "admin123"
print(password)

# After (transduced)
password = os.getenv("PASSWORD")
# [CHOMSKY] sensitive output removed
```

Built with `pyformlang`. The transducer is structure-preserving: it rewrites only flagged lines, leaving the rest of the file untouched.

Module 4 — Validation

Context-Free Grammar (CFG)

4-tuple grammar $G = (V, T, P, S)$:

```
<ConfigFile>  ::= <Section>+
<Section>     ::= ID "{" <Entry>+ "}"
<Entry>       ::= <Assignment> | <Section>
<Assignment>  ::= ID "=" <Value> ";"
<Value>       ::= <EnvRef> | STRING | INT | BOOL
<EnvRef>      ::= "${" ID "}"
```

Why not regular? Matched nested braces require unbounded memory. By the Pumping Lemma, no finite automaton can recognize $\{^n\ w\ \}^n$ — a CFG (and pushdown automaton) is strictly necessary. Built with `textX`, the validator also enforces that sensitive keys use `${ENV_VAR}` references.

Pipeline Architecture

Results

162 UNIT TESTS · **4** MODULES · **100%** PASSING

Module	Tests	Avg. runtime
Detector	43	0.02 s
Classifier	43	0.05 s
Transformer	48	0.12 s
Validator	28	0.04 s

Conclusions

- Each **level of the Chomsky hierarchy** maps naturally to a layer of security analysis: lexical (regex), behavioral (DFA), rewriting (FST), structural (CFG).
- State tracking and structural validation reduce false positives compared to pure-regex scanners.
- The CFG layer catches nested misconfigurations that no regex-only tool can detect.

Future Work · References

Extend CFG to YAML / TOML · Git pre-commit hooks · Go, Rust, Java support · ML-assisted token classification.

[1] Sipser, *Introduction to the Theory of Computation*, 3rd ed. · **[2]** Hopcroft, Motwani, Ullman, *Automata Theory, Languages, and Computation.* · **[3]** pyformlang · **[4]** textX · **[5]** Gitleaks, TruffleHog.